Tutorial: Attaching your own data source

For developers with no prior experience · ~30 minutes · runs entirely against the demo scenario · every step with an example

1 The model in three terms

Everything in the sources framework revolves around three things. **Record**: the normalised result — the report only ever sees records, never your system. For an SLO:

```
{"service": "Order API", "slo": "availability",
 "target_pct": 99.9, "sli_pct": 99.95,
 "window": "30d", "source": "csv:slos.csv"}
```

Provider: a translator — it reads your system (a file, a REST API, anything) and returns records; one Python file, one class. **Register**: the JSON file a fetch writes; the solution config references it ("slo": "slo.json"), the report renders it. One rule keeps everything comparable: **providers never judge** — whether an SLO is breached is decided centrally, by the same rule for every source.

2 Use it before you build it (5 minutes)

Generate the demo portfolio, list the providers, and run your first fetch with the file provider — the result (my_slo.json) is the target format of every source:

```
python -m testdata_generator --scenario portfolio --output demo --seed 42
python -m sources providers

echo {"provider": "file", "path": "demo/slo_alpha.json"} > my_sources.json
python -m sources fetch --kind slo --config my_sources.json --output my_slo.json
```

3 Build your own provider, step by step

We attach a CSV file (teams often keep SLO values in a spreadsheet) — no server needed, and the pattern is identical for any system. The finished reference lives in sources/providers/csv_slo.py. **Step 1:** create ONE file, sources/providers/my_csv.py:

```
from __future__ import annotations

import csv
from datetime import datetime
from pathlib import Path

from sources.base import KIND_SLO, SloRecord


class MyCsvSource:
    # (1) Name in Configs und in der providers-Liste / name in configs
    provider_id = "my_csv"
    # (2) Was diese Quelle liefern kann / what it can deliver
    kinds = (KIND_SLO,)

    # (3) Der Uebersetzer / the translator
    def fetch(self, kind, config, log):
        path = Path(config["path"])
        if not path.is_file():
            raise RuntimeError(f"my_csv: '{path}' missing.")
        fetched_at = datetime.now().isoformat(timespec="seconds")
        records = []
        with path.open(encoding="utf-8-sig", newline="") as f:
            for row in csv.DictReader(f):
                records.append(SloRecord(
                    service=row["service"],
                    slo=row["slo"],
                    target_pct=float(row["target_pct"]),
                    sli_pct=float(row["sli_pct"]) if row.get("sli_pct") else None,
                    source=f"my_csv:{path.name}",
```

```
                fetched_at=fetched_at,
            ))
    log(f"  my_csv: {len(records)} records")
    return records


# (4) Genau dieses Objekt sucht die Auto-Discovery / discovery hook
PROVIDER = MyCsvSource()
```

Four numbered ideas are the whole contract: an id, the kinds, a fetch() that translates, and a PROVIDER object. No registry to edit anywhere — the file's existence registers it. **Step 2:** python -m sources providers → 'my_csv: slo' appears. **Step 3:** feed it data (team_slos.csv):

```
service,slo,target_pct,sli_pct
Order API,availability,99.9,99.95
Checkout,availability,99.5,99.55
```

Steps 4 and 5: create the fetch config, fetch, reference the register in the solution config ("slo": "my_slo.json") and render the report — the 'Service Levels & Error Budgets' section shows your CSV rows, the Data source column reads my_csv:team_slos.csv:

```
{"provider": "my_csv", "path": "team_slos.csv"}

python -m sources fetch --kind slo --config csv_source.json --output my_slo.json
python -m portfolio demo/solution_alpha.json --output report.html --browser
```

4 A REST system instead of a file?

Same pattern, only fetch() changes — the shared helper handles HTTP, auth and error mapping:

```
from sources.http import bearer, get_json, token_from_env

def fetch(self, kind, config, log):
    headers = bearer(token_from_env(config, "MY_SYSTEM_TOKEN"))
    data = get_json(config["base_url"] + "/api/slos", headers, "MySystem")
    return [SloRecord(service=e["name"], slo=e["goal"],
                      target_pct=e["target"], sli_pct=e["current"],
                      source="my_system") for e in data]
```

• Tokens only from environment variables (token_from_env) — never in configs, never in logs.

• get_json maps errors: 401/403 points at the possibly missing API approval AND at the file path as fallback.

• Test with mocks, not against the live system — copy the request-recorder pattern from tests/sources/unit/test_rest_providers.py.

5 Swapping a source

Swapping means: change the fetch config, nothing else. Register name, solution config and report stay untouched — only the Data source column shows the new origin:

```
# vorher / before:
{"provider": "my_csv", "path": "team_slos.csv"}

# nachher / after (gleicher fetch-Befehl, gleiches Register):
{"provider": "prometheus", "base_url": "https://prom.intern",
 "services": [{"service": "Order API", "slo": "availability",
               "target_pct": 99.9,
               "sli_query": "avg_over_time(up[30d])", "scale": 100}]}
```

6 Combining two sources

A config may hold a sources list; the records merge into ONE register, each row keeping its origin. Typical: most services in monitoring, two legacy systems in a spreadsheet:

```
{"sources": [
  {"provider": "prometheus", "base_url": "https://prom.intern",
   "services": [ ... ]},
  {"provider": "my_csv", "path": "team_slos.csv"}
]}
```

7 Managing sources day to day

• Inventory: python -m sources providers is the authoritative list — a new file appears automatically.

• Tokens: one environment variable per system (PROMETHEUS_TOKEN, GITHUB_TOKEN, GITLAB_TOKEN, SONAR_TOKEN; your own via token_env).

• Conventions: one file per source; providers translate, never judge; unmeasured values are None, not a guess.

• Errors: 401/403 carries the approval hint — until it arrives, ship the same data via file or csv.

• Tests: a small test file per provider (happy path, one broken input, a readable error message).

8 Checklist for a new source

• One file in sources/providers/ with provider_id, kinds, fetch(), PROVIDER — providers lists it.

• fetch() returns normalised records; every record sets source; no judgement in the provider.

• Tokens via token_from_env; mock tests green; swapping back to file/csv still works (the fallback while approvals are pending).